

Weight [SimTalk] - data type

A *local variable* of data type *weight* has a maximum range of values between $-8.9 \cdot 10^{307} \leq \text{weight} \leq 8.9 \cdot 10^{307}$ which is displayed like this $-8.9\text{e}307 \leq \text{real} \leq 8.9\text{e}307$.

Remarks

When assigning it to a variable or an attribute, *Plant Simulation* interprets the value as kilograms (kg). When outputting it, *Plant Simulation* converts the value to the unit you selected under **File > Model Settings/Preferences > Units > Mass**.

The data types *time*, *length*, *weight*, *speed*, and *acceleration* are not compatible! You can, for example, only assign a value of data type *length*, *real*, or *integer* to a variable of data type *length*.

See also

[isWeight \[SimTalk\]](#)

Instantiating Local Lists and Tables

Instantiating Local Lists and Tables

A *local variable* of data type **list**, **queue**, **stack**, or **table** and lists within lists and tables within tables does not contain the data itself, but a reference to the respective data structure.

When the variables are entered into a *Method*, they are *void*, i.e., they do not contain a reference to a data structure yet. You have to create the variable, before you can access the data.

Lists and tables provide the *methods* **create** and **createNestedList** for instantiating them.

create [SimTalk] - local list

Creates an empty data structure without contents in the *local variable* designated by <Local-variable>.

Remarks

create applies to *local variables* of data type [list](#), [queue](#), [stack](#), and [table](#).

Type

Method

Syntax

```
<Local-variable>.create([NumberOfRows:integer])
```

Parameter

The optional parameter *NumberOfRows* of data type *integer* designates the number of rows in the list or table.

Example

```
var orderlist: table[string,real]
orderlist.create
orderlist[1,1] := "cans"
orderlist[2,1] := 3000.0
orderlist.forget // destroys the table
orderlist.create(4) // recreates the table with 4 rows
```

See also

[Data Types in Local Variables](#)

createNestedList [SimTalk] - local variable

Creates a new sublist or subtable in the *local variable* designated by <local-list> or by <local-table>.

Remarks

The local variable has to be of data type **list**, **queue**, **stack**, or **table**. The data type of the column of this list or table has to be of data type *list*, *stack*, *queue*, or *table* as well.

Type**Method****Syntax**

```
<Local-list>.createNestedList(Column:integer, Row:integer[, Name:string])
→ list
<Local-table>.createNestedList(Column:integer, Row:integer[, Name:string])
→ list
```

Parameters

You can specify the following parameters:

- The parameter *Column* of data type *integer* designates the column of the cell.
- The parameter *Row* of data type *integer* designates the row of the cell.
- The optional parameter *Name* of data type *string* designates the name of the sublist or subtable to be created.

Note

This method also applies to sublists and subtables!

Return Value

The return value has the data type *list*.

Is the nested list (*DataList*, *DataQueue*, *DataStack*, or *DataTable*) that was created.

Examples

```
var t: table[table]
t.create           // creates a table
```

```
t.createNestedList(1,1) // creates a subtable
t[1,1][2,3] := "Hello world" // writes Hello world into the nested table
```

The code inserts a subtable into cell 1 of the our table. It then inserts Hello world into the cell at the position 2,3.

The screenshot shows two overlapping windows from the SimTalk IDE. The background window, titled '.Models.Model.Method.t', displays a table named 'table1'. It has a header row with 'table' and '1', and a data row with '1' and 'table1'. The foreground window, titled '.Models.Model.Method.t[1,1]', displays a subtable. It has a header row with 'string 1', 'string 2', and 'string 3'. The data rows are: row 1 (empty), row 2 (empty), row 3 (empty, 'Hello world', empty), row 4 (empty), and row 5 (empty).

```
var subtable: table
MyDataTable.createNestedList(2,3) // creates a subtable
subtable := MyDataTable[2,3]
subtable.createNestedList(1,1) // creates a subtable within a subtable
subtable[1,1][4,4] := "Hello world"
```

See also

[setCommonFormat \[SimTalk\]](#)

[Activating and Deactivating Common Format](#)

[createNestedList \[SimTalk\] - DataQueue](#)

[createNestedList \[SimTalk\] - DataList](#)

[createNestedList \[SimTalk\] - DataTable](#)